Instituto Tecnológico y de Estudios Superiores de Monterrey

Maestría en Inteligencia Artificial:

Spin Voyager & Spin Compass: Avance 5. Modelo final

Tania Alicia Caballero Saavedra - A01795957

Oscar Enrique García García - A01016093

Dante Rosas Fragoso – A01795850

Profesor Titular: Dra. Grettel Barceló Alonso

Asesor: Dr. Horacio Martínez Alfaro

Proyecto Integrador

Marzo, 2026

Índice

Introducción . . . 1

¿Es viable un modelo Ensemble para Spin Compass? . . . 2

- Evolución de Spin Compass: Hacia un Enfoque de Ensemble Dinámico . . . 3
- Posibles configuraciones Ensemble aplicables a Spin . . . 3
  - Ensemble de Verificación (Generative + Judge Model) . . . 4
  - Ensemble por Routing Inteligente (Orquestación Decisional) . . . 4
  - Ensemble Determinístico + Generativo (Híbrido Simbólico) . . . 5
  - *Ensemble* Multimodal (Visión + Texto) . . . 5
- Viabilidad, Impacto y Evaluación Estratégica del Enfoque Ensemble . . . 5
  - Análisis de Impacto en Métricas Clave . . . 6
  - Factibilidad Técnica en Google Cloud Platform . . . 6

Transición Arquitectónica: Adopción de LangChain y Deprecación del NoiseReducer . . . 7

- Motivación para la Decisión . . . 7
- Justificación Técnica . . . 8
  - Estandarización del Preprocesamiento . . . 8
  - Reducción de Deuda Técnica . . . 8
  - Orquestación Integral . . . 9
- Análisis de Riesgo . . . 9
- Beneficios Estratégicos . . . 10
- Cambio de Paradigma . . . 11
  - Acciones Tácticas para la Implementación . . . 11

LangChain como Habilitador de un Ensemble Condicional Futuro . . . 12

- Marco Conceptual . . . 12
- Limitaciones del Pipeline Lineal . . . 12
- Orquestación como Base para Ensemble Condicional . . . 13
  - El Paradigma del Ensemble Condicional . . . 14
- Ventajas Estratégicas de un *Ensemble* Condicional . . . 14

Evolución Progresiva Propuesta 15
El Rol de *LangChain* como Catalizador 16
Aplicación para SLACK 17
Descripción de Componentes y Arquitectura 18
Flujo de Datos (Alto Nivel) 19
Estatus Actual 19
Pruebas de Funcionalidad de Compass 20
Evaluación del Conocimiento del Agente 20
Consulta sobre Políticas de Viáticos 21
Consulta Inicial 21
Aclaración del Puesto 23
Consulta sobre Monto Máximo de Propina 24
Validación de Enlaces y Anexos 24
Prueba de Privacidad y Acceso a Información Personal 25
Sistema de Feedback 26
Feedback Positivo 26
Feedback Negativo 27
Funcionalidades del Administrador 28
Configuración del Agente en Slack 28
Configuración de Ámbitos (Scopes) de la API 29
Spin Bridge – Dashboard Ejecutivo 31
Dashboard Principal 31
Análisis de Conversaciones con Feedback Negativo 31
Configuración Avanzada del Agente (En Desarrollo) 32
Estado del Proyecto y conclusiones 33
Bibliografía 34

Introducción

La arquitectura actual de **Spin Compass** ha demostrado que un enfoque basado en *Retrieval-Augmented Generation* (RAG) es viable y efectivo para garantizar integridad documental dentro del ecosistema de Finanzas y alguna otra área. Sin embargo, conforme el sistema transita desde una fase experimental hacia una arquitectura productiva y escalable, surge una necesidad clara: evolucionar desde un RAG funcional hacia un RAG orquestado y modular.

En los avances anteriores, la optimización se centró principalmente en la calidad del dato (**NoiseReducer 2.0**), la segmentación semántica (**Max-Min Semantic Chunking**) y la eficiencia del modelo generativo (**Gemini 2.0 Flash-Lite**). No obstante, la creciente complejidad operativa del sistema —incluyendo gestión de memoria conversacional, control de tokens, integración con bases vectoriales y cumplimiento de restricciones de infraestructura en **Google Cloud Platform**— hace evidente que el siguiente paso no es únicamente mejorar componentes individuales, sino introducir una capa formal de orquestación.

En este contexto, el uso de frameworks especializados como **LangChain** se perfila como una evolución natural del proyecto. Estas bibliotecas permiten desacoplar explícitamente los distintos módulos del *pipeline* (embeddings, retriever, memory, prompt templates, herramientas externas y modelos generativos), transformando un flujo lineal en un sistema componible y extensible.

La incorporación de un framework de orquestación no implica sustituir la lógica desarrollada en los avances previos, sino encapsularla dentro de una arquitectura más robusta que permita:

- Gestión estructurada de *retrievers* y herramientas externas.
- Implementación de agentes capaces de decidir dinámicamente entre RAG tradicional y *Model-Assisted Generation* (MAG).
- Control fino de memoria conversacional en **Slack**.
- Observabilidad y trazabilidad del flujo de inferencia.

- Optimización del consumo de tokens mediante encadenamiento selectivo de componentes.
- Integración nativa con servicios de **Google Cloud** como **AlloyDB**, **Vertex AI** y **Cloud Run**.

Desde una perspectiva técnica, el *stack* de dependencias adoptado permite construir una arquitectura donde el LLM deja de ser un componente monolítico y se convierte en un nodo dentro de un grafo de ejecución controlado. Esto habilita estrategias avanzadas como *routing* dinámico de consultas, *multi-step reasoning* controlado y ensamblaje híbrido de modelos.

El **Avance 5**, por tanto, no se enfoca únicamente en las métricas actuales, sino en sentar las bases de una arquitectura sostenible a largo plazo. La meta es transformar **Spin Compass** en un sistema modular, auditable y escalable, donde cada componente pueda evolucionar de manera independiente sin comprometer la integridad del conjunto.

En síntesis, la incorporación de una biblioteca de orquestación representa el paso estratégico que permite consolidar la transición desde un RAG optimizado experimentalmente hacia una infraestructura de inteligencia conversacional empresarial lista para producción.

¿Es viable un modelo Ensemble para Spin Compass?

La evolución natural de sistemas *Retrieval-Augmented Generation* (RAG) empresariales plantea una pregunta estratégica: ¿es suficiente un único modelo generativo para garantizar precisión, robustez y eficiencia operativa, o es conveniente implementar un enfoque de tipo *ensemble*? En el contexto de **Spin Compass**, esta cuestión trasciende lo teórico y se convierte en una decisión arquitectónica con implicaciones directas en costos, latencia, confiabilidad y gobernanza del conocimiento.

Evolución de Spin Compass: Hacia un Enfoque de Ensemble Dinámico

La arquitectura actual de **Spin Compass** —basada en RAG con **Max-Min Semantic Chunking**, base vectorial en **AlloyDB** y el modelo **Gemini 2.0 Flash-Lite**— está lista para evolucionar de un flujo lineal hacia un grafo de ejecución dinámico basado en principios de *Ensemble*. Mientras que en el aprendizaje automático tradicional un *ensemble* busca reducir la varianza combinando modelos, en este sistema la estrategia se centra en la eficiencia operativa y la especialización.

Dadas las restricciones críticas de latencia (< 5 segundos) y los costos en **Google Cloud Platform**, el sistema prioriza un *Ensemble* Funcional y Híbrido por encima de la redundancia generativa. Esto implica que las consultas interactúen con un ecosistema donde coexisten modelos especializados y reglas determinísticas (como los validadores de políticas y la limpieza avanzada de **NoiseReducer 2.0**). Al integrar estas capacidades simbólico-generativas, se optimiza el uso de **Vertex AI** y se mejora la precisión medida por el *Golden Dataset*, elevando métricas clave como el RCC, *Hit Rate* y MRR.

Posibles configuraciones Ensemble aplicables a Spin

Es fundamental precisar que, dadas las restricciones cronológicas del proyecto y el *stack* tecnológico definido, la implementación actual no profundizará en la ejecución técnica de estas arquitecturas. El desarrollo presente se centra en establecer una base operativa fundamental.

No obstante, estas configuraciones se integran en este documento como una hoja de ruta estratégica. Su inclusión permite que **Spin** disponga de un marco de escalabilidad probable, sirviendo como punto de partida técnico para futuras optimizaciones que busquen elevar la robustez del sistema en fases posteriores de producción.

Ensemble de Verificación (Generative + Judge Model)

Esta configuración introduce una capa de auditoría en tiempo real para asegurar la integridad de la respuesta. El flujo no termina con la generación de **Gemini 2.0 Flash-Lite**, sino que somete el resultado a un «Modelo Juez» (*Self-Correction Loop*). Este segundo agente valida tres pilares:

- Atribución: ¿La cita existe en el *chunk*?
- Fidelidad: ¿Se inventaron datos?
- Consistencia: ¿Contradice la política recuperada de **AlloyDB**?

Impacto en Negocio: Es vital para mitigar alucinaciones en políticas financieras donde un error de interpretación tiene consecuencias legales.

Análisis de Viabilidad: Debido a que aumenta el consumo de tokens, su implementación debe ser selectiva. Se recomienda activar este *ensemble* solo cuando la consulta detecte términos de alta sensibilidad o ambigüedad técnica.

Ensemble por Routing Inteligente (Orquestación Decisional)

En lugar de procesar cada entrada con el mismo *pipeline* pesado, se implementa un *Router* de Intenciones liviano a la entrada del grafo. Este modelo actúa como un clasificador de tráfico que decide el camino óptimo:

- Consultas Simples: Flujo RAG estándar.
- Consultas Complejas: Activa el modo MAG (*Multi-Agent Generation*) o contextos extendidos si se requiere comparar múltiples políticas.
- Interacciones Básicas: Respuestas directas para saludos o FAQs administrativas, realizando un *bypass* del proceso de *retrieval* en **AlloyDB**.

Viabilidad: Máxima. Es la estrategia más eficaz para mantener la latencia por debajo de los 5 segundos y reducir el costo operativo por usuario.

Ensemble Determinístico + Generativo (Híbrido Simbólico)

Representa la configuración más sólida y alineada con la arquitectura actual de **Spin Compass**. Aquí, el LLM no opera en el vacío, sino que está «escoltado» por un motor de reglas previo. Antes de que **Gemini** sintetice la información, el sistema ejecuta validadores *Regex* para detectar PII (datos personales), aplica **NoiseReducer 2.0** para limpiar el contexto y consulta reglas de negocio que filtran versiones obsoletas de documentos.

- Seguridad y Cumplimiento: El motor simbólico decide si una política está vigente basándose en metadatos duros, dejando al LLM únicamente la tarea de redacción y síntesis.

Viabilidad: Estratégica. Reduce drásticamente el riesgo legal y garantiza que la salida sea técnicamente impecable.

Ensemble Multimodal (Visión + Texto)

Aunque no es una prioridad inmediata, esta ruta define la escalabilidad a futuro de **Spin Compass**. Permitiría al sistema no solo «leer» texto plano, sino interpretar PDFs escaneados, imágenes de facturas o formularios manuscritos mediante modelos de visión de la familia **Gemini**.

- Evolución: La arquitectura modular actual facilita que un codificador de visión se integre como una rama adicional del grafo de ejecución, permitiendo que la información visual se convierta en *embeddings* vectoriales compatibles con el flujo de **AlloyDB**.

Viabilidad, Impacto y Evaluación Estratégica del Enfoque Ensemble

La implementación de una arquitectura *Ensemble* en **Spin Compass** no debe entenderse como una ejecución constante, sino como una activación condicional basada en la complejidad de la consulta. Una aplicación indiscriminada elevaría los costos operativos sin garantizar un beneficio proporcional; por el contrario, un diseño orientado a la eficiencia permite equilibrar la precisión con la sostenibilidad del sistema.

Análisis de Impacto en Métricas Clave

Un *Ensemble* bien orquestado transforma el perfil de rendimiento del sistema. Mientras que métricas de recuperación como el *Hit Rate* se mantienen neutrales, se esperan mejoras significativas en la confiabilidad y la calidad de la respuesta:

Métrica	Impacto Esperado	Justificación
Faithfulness	Mejora Significativa	El modelo juez reduce drásticamente las alucinaciones y errores factuales.
RCC / MRR	Incremento Leve	El *re-ranking* y la validación refinan la relevancia y precisión final.
Latencia	Riesgo de Aumento	Requiere optimización constante para no superar el umbral de 5 segundos.
Costo	Variable	Depende estrictamente de una política de *Routing* inteligente y condicional.

Tabla 1: Impacto esperado de la arquitectura *Ensemble* en métricas clave

Factibilidad Técnica en Google Cloud Platform

Desde la perspectiva de infraestructura, **Spin Compass** cuenta con una base sumamente compatible para esta evolución. La arquitectura sobre **GCP** facilita la transición sin necesidad de reestructuraciones profundas:

- **Orquestación:** Herramientas como **LangChain** o **Vertex AI Pipelines** permiten gestionar el flujo entre modelos de manera fluida.
- **Escalabilidad: AlloyDB** y **Cloud Run** soportan la ejecución asíncrona necesaria para procesos de verificación en paralelo.
- **Modelado: Vertex AI** ofrece acceso nativo a la familia **Gemini**, permitiendo alternar entre modelos **Flash** (velocidad) y **Pro** (razonamiento profundo) según el segmento del *ensemble*.

Para el éxito de **Spin Compass**, la pregunta fundamental no es si es posible implementar un *Ensemble*, sino bajo qué criterios activarlo. El desafío principal no es técnico, sino de **gobernanza y optimización económica**. Una implementación condicionada por el tipo de consulta, el nivel de ambigüedad o el riesgo financiero asociado permitirá elevar la confiabilidad documental a niveles críticos sin comprometer la sostenibilidad del proyecto.

Transición Arquitectónica: Adopción de LangChain y Deprecación del NoiseReducer

Motivación para la Decisión

Los avances anteriores demostraron que la calidad del preprocesamiento documental influye directamente en las métricas de recuperación (*Hit Rate*, MRR y RCC). La implementación de **NoiseReducer 2.0** permitió remover ruido estructural, mejorar la coherencia semántica de los fragmentos y optimizar el uso de tokens en **Vertex AI**.

Sin embargo, conforme **Spin Compass** evoluciona hacia una arquitectura empresarial escalable, surge un **principio rector de ingeniería**:

«No es suficiente que un sistema funcione; debe ser mantenible, extensible y estandarizado.»

El componente **NoiseReducer**, aunque efectivo, representa una solución altamente personalizada, acoplada al dominio actual y dependiente de mantenimiento manual continuo. Este enfoque, a largo plazo, introduce **deuda técnica**. En consecuencia, se propone la adopción formal de **LangChain** como *framework* de orquestación, reemplazando gradualmente el componente independiente.

Justificación Técnica

Estandarización del Preprocesamiento

La adopción de **LangChain** permite transformar el preprocesamiento de un conjunto de *scripts* aislados a un ***pipeline* de ingesta industrializado**. Al aprovechar herramientas como los *document loaders* estructurados y los *text splitters* configurables, el sistema logra una integración directa entre la carga de datos y los procesos de *retrieval*.

En lugar de depender de un mantenimiento extensivo de reglas *regex* personalizadas, la estrategia evolució hacia el uso de cargadores especializados para formatos PDF, TXT y Markdown. Bajo este nuevo esquema, las transformaciones manuales se limitan a casos estrictamente necesarios, delegando la integridad estructural al propio flujo de *chunking* y recuperación. Esta transición no elimina el preprocesamiento, sino que lo **encapsula** dentro de una arquitectura modular que facilita la mantenibilidad ante nuevas categorías documentales.

Reducción de Deuda Técnica

La transición hacia **LangChain** aborda los desafíos de escalabilidad inherentes al mantenimiento manual. En el esquema anterior, la limpieza de datos exigía una validación continua de patrones de ruido y el ajuste manual ante cambios en los formatos de las políticas; un esfuerzo operativo que escala linealmente con el volumen de información.

Al migrar a una arquitectura estandarizada, **Spin Compass** logra:

- **Reutilizar componentes robustos** mantenidos por la comunidad global.
- **Reducir la dependencia** de reglas específicas de dominio.
- **Facilitar el *onboarding*** de nuevos desarrolladores al sistema.

Más allá de la simplificación del código, este cambio habilita una integración nativa con herramientas avanzadas como sistemas de memoria y agentes autónomos, transformando el preprocesamiento en una base ágil preparada para el **crecimiento estratégico**.

Orquestación Integral

El componente **NoiseReducer**, aunque efectivo en su concepción original, presenta una limitación crítica: su operabilidad se restringe exclusivamente a la fase de preprocesamiento. Al funcionar como un módulo estático y aislado, carece de visibilidad sobre el ciclo de vida completo de la consulta, lo que impide que el sistema reaccione a ruidos o inconsistencias que emergen durante la recuperación o la síntesis.

La adopción de **LangChain** rompe el paradigma del flujo lineal (*Input* → **NoiseReducer** → RAG → *Output*) para dar paso a un **grafo de ejecución dinámico**. Bajo este nuevo esquema, **Spin Compass** adquiere capacidades avanzadas que antes eran inviables:

- **Orquestación Integral:** Fusión nativa entre la memoria del usuario, la recuperación de documentos y la generación de respuesta.
- **Validaciones Encadenadas:** Capacidad de verificar la calidad de los datos en múltiples puntos de control del *pipeline*.
- ***Routing* y Verificación Condicional:** Implementación de un ***Ensemble* Estratégico** donde el sistema decide la ruta de procesamiento según la complejidad de la consulta.

Esta evolución marca el fin del módulo de limpieza como un componente externo, transformándolo en una **propiedad intrínseca** de una arquitectura adaptable y escalable.

Análisis de Riesgo

La eliminación del módulo **NoiseReducer** conlleva desafíos técnicos que deben gestionarse con rigor analítico. El principal riesgo identificado es la **reintroducción de ruido estructural** en los contextos recuperados, lo que podría derivar en una dependencia excesiva del modelo generativo y provocar una ligera reducción inicial en la métrica RCC.

No obstante, esta transición se sustenta en una **estrategia de mitigación** de cuatro pilares:

1. **Implementación de *Splitters* Semánticos:** El uso de segmentadores avanzados compensa la ausencia de limpieza previa, garantizando cohesión lógica en los fragmentos.

2. **Limpieza Quirúrgica:** Se aplica una normalización mínima pero suficiente dentro del *pipeline*, eliminando el ruido crítico sin la rigidez de un módulo aislado.
3. **Validación con *Golden Dataset*:** Cada cambio se contrasta contra el conjunto de datos de referencia, asegurando que la precisión no descienda de los umbrales de producción.
4. **Experimentación Controlada:** La decisión final se basa en evidencia empírica mediante la evaluación comparativa de métricas pre y post-migración.

En última instancia, este enfoque garantiza que la evolución hacia una arquitectura más ágil no comprometa la integridad de las respuestas, manteniendo la fiabilidad de **Spin Compass** bajo un **control de calidad constante**.

Beneficios Estratégicos

La transición genera beneficios claros:

Dimensión	Impacto Esperado
Escalabilidad	Alta
Mantenibilidad	Alta
Integración futura	Muy alta
Reducción de deuda técnica	Significativa
Dependencia de reglas manuales	Disminuye sustancialmente
Gobernanza y trazabilidad	Mejora integral

Tabla 2: Impacto esperado de la transición arquitectónica por dimensión

La reestructuración del sistema no solo optimiza el flujo actual, sino que sienta las bases para implementar capacidades críticas de próxima generación:

- ***Routing* Dinámico de Consultas:** El sistema adquiere la facultad de clasificar la intención y complejidad del usuario, dirigiendo cada solicitud hacia la ruta de procesamiento más eficiente.

- ***Ensemble* Condicional:** Permite la activación selectiva de modelos especializados solo cuando la ambigüedad lo requiere, garantizando precisión sin comprometer la sostenibilidad económica.
- ***Multi-step Reasoning* Controlado:** Facilita la descomposición de consultas complejas en sub-tareas lógicas para abordar comparativas entre múltiples políticas o normativas.
- **Observabilidad del *Pipeline* Completo:** Al operar bajo una estructura de grafo, cada nodo genera trazas detalladas que permiten auditar el comportamiento y asegurar la transparencia en la gobernanza de datos.

Cambio de Paradigma

El módulo **NoiseReducer** representaba un enfoque de «control manual exhaustivo», donde la calidad dependía de reglas rígidas. La adopción de **LangChain** implica una transición conceptual profunda:

> **De:** *Un sistema optimizado mediante reglas determinísticas extensivas que requieren mantenimiento constante.*
>
> **A:** *Un sistema orquestado mediante componentes especializados, donde la robustez emerge de una arquitectura modular y adaptable.*

En esta nueva etapa, se prioriza el diseño sistémico sobre la micro-optimización textual. La inteligencia del *pipeline* no reside en la limpieza previa de cada carácter, sino en la capacidad de la arquitectura para gestionar el flujo de información de manera integral.

Acciones Tácticas para la Implementación

Para materializar esta visión, se han definido los siguientes pasos:

1. **Depreciación del** NoiseReducer**:** El módulo deja de ser un componente central para evitar la rigidez operativa y la deuda técnica.
2. **Migración de Ingesta y *Chunking*:** El flujo de datos se traslada hacia la infraestructura de **LangChain**, aprovechando su segmentación semántica nativa.

3. **Validaciones Críticas Residuales:** Se mantienen únicamente intervenciones indispensables (como la detección de PII), delegando el resto al orquestador.
4. **Evaluación de Impacto:** Validación rigurosa mediante métricas (**Hit Rate**, MRR, RCC y *Faithfulness*) para asegurar la integridad del sistema.
5. **Arquitectura Orientada a Agentes:** Preparación del *pipeline* para una evolución hacia modelos de agentes inteligentes en el futuro cercano.

LangChain como Habilitador de un Ensemble Condicional Futuro

Marco Conceptual

La decisión de adoptar un *framework* de orquestación como **LangChain** no constituye en sí misma un modelo *ensemble*. Sin embargo, representa un paso estructural necesario para habilitar arquitecturas *ensemble* de manera controlada, sostenible y económicamente viable en etapas posteriores del proyecto.

Un sistema RAG monolítico puede optimizarse internamente, pero su capacidad de evolución es limitada si no se encuentra encapsulado dentro de una arquitectura modular. La implementación de **LangChain** transforma el *pipeline* lineal en un grafo componible, donde distintos nodos pueden activarse condicionalmente según criterios de riesgo, ambigüedad o complejidad de la consulta.

LangChain no es el *ensemble*, pero sí la infraestructura que lo hace viable.

Limitaciones del Pipeline Lineal

En su configuración actual (pre-orquestación avanzada), el flujo opera bajo una lógica determinística:

$$[\text{Consulta}] \rightarrow [\text{Retriever}] \rightarrow [\text{Modelo Generativo}] \rightarrow [\text{Respuesta}]$$

Este esquema presenta dos restricciones fundamentales:

1. **Homogeneidad de Procesamiento:** Todas las consultas reciben el mismo tratamiento computacional, independientemente de su simplicidad.
2. **Rigidez de Razonamiento:** No existe un mecanismo explícito para escalar la profundidad de análisis según el nivel de riesgo o sensibilidad de la información.

Para entornos corporativos, esta homogeneidad puede resultar subóptima, ya que no permite diferenciar entre una consulta administrativa trivial y una interpretación compleja de normativas financieras.

Orquestación como Base para Ensemble Condicional

Con la adopción de *LangChain* como orquestador, el *pipeline* de **Spin Compass** trasciende la ejecución lineal para reestructurarse como un grafo de flujo dinámico. En esta arquitectura, la consulta no sigue un camino predefinido, sino que es evaluada en cada nodo para determinar la ruta óptima de procesamiento.

El flujo se desglosa en etapas críticas de decisión que garantizan precisión y eficiencia:

- *Clasificador de Intención (Router):* Un modelo ligero identifica si la consulta es transaccional, informativa o comparativa, asignándola al canal adecuado desde el inicio.
- *Evaluación de Score de Recuperación:* Tras el *retrieval* en **AlloyDB**, se analiza la relevancia semántica de los *chunks*. Si el *score* es bajo, el sistema puede decidir ampliar la búsqueda o pivotar la estrategia.
- *Lógica de Decisión Condicional:* Aquí es donde el sistema elige su «modo de combate» según la complejidad detectada:
  - **RAG Simple:** Para consultas directas con alta confianza de recuperación.
  - **RAG + Verificación (Judge Model):** Se activa ante consultas sobre políticas críticas para asegurar la ausencia de alucinaciones.
  - **Modo MAG (Multi-Agent Generation / Long Context):** Para solicitudes que requieren contrastar múltiples documentos o analizar contextos extensos.

- *Escalamiento a Soporte Humano:* Un mecanismo de seguridad que se activa cuando el sistema detecta que la consulta supera el umbral de confianza o cae fuera del dominio de conocimiento.

El Paradigma del Ensemble Condicional

Bajo este modelo, emerge el verdadero potencial de un *Ensemble* Condicional. A diferencia de los sistemas tradicionales donde todos los componentes se ejecutan en paralelo, aquí la arquitectura se rige por principios de eficiencia selectiva:

- *Ejecución On-Demand:* Los modelos de verificación o síntesis compleja solo se invocan cuando la consulta lo amerita, eliminando la redundancia innecesaria.
- *Activación por Métricas Objetivas:* El sistema utiliza indicadores como el *confidence score* o la detección de entidades sensibles para disparar ramas específicas del grafo.
- *Control Dinámico de Recursos:* El costo computacional se gestiona en tiempo real, priorizando el gasto de tokens solo en interacciones de alto valor.
- *Optimización de Latencia:* Al evitar procesos pesados en consultas triviales, el sistema mantiene el tiempo de respuesta dentro de los umbrales de < 5 segundos para la mayoría de los usuarios.

Ventajas Estratégicas de un *Ensemble* Condicional

La implementación de esta estructura orquestada por *LangChain* permite consolidar cuatro pilares fundamentales para la operatividad de **Spin Compass**:

A) Gobernanza Semántica Las consultas categorizadas como de «alto riesgo» activan automáticamente capas de validación adicional. Esto reduce drásticamente la probabilidad de errores factuales o alucinaciones en decisiones financieras críticas.

B) Optimización de Costos Se elimina la redundancia innecesaria. El modelo verificador o el uso de modelos de mayor capacidad (*pro*) solo se activan cuando los umbrales de ambigüedad definidos en el *router* así lo ameritan.

C) Escalabilidad Controlada La arquitectura modular permite que, al incorporar nuevas categorías documentales (onboarding, beneficios, TI), el componente de *routing* se expanda de forma aislada sin requerir un rediseño del sistema completo.

D) Observabilidad Arquitectónica Al operar como un grafo de ejecución, cada paso de la decisión es trazable. El flujo queda documentado y es auditable, cumpliendo con los estándares de transparencia exigidos en entornos corporativos.

La clave estratégica reside en que el *ensemble* deja de ser una redundancia permanente para convertirse en un mecanismo adaptativo de alta precisión.

Evolución Progresiva Propuesta

La evolución de **Spin Compass** hacia un sistema de inteligencia adaptativa no es un evento único, sino un proceso incremental diseñado para mitigar riesgos técnicos y financieros. Esta transición se estructura en tres fases críticas:

1. **Fase 1 – Orquestación Modular y Estabilización:** El objetivo primordial es la migración del *pipeline* lineal actual hacia una estructura de grafos con *LangChain*. Esta fase establece los cimientos de «conectividad» entre componentes.
   - *Routing* Básico: Implementar la capacidad de bifurcar consultas simples de las complejas.
   - *Baseline* de Métricas: Establecer una línea base post-migración para asegurar que la orquestación no degrade la latencia por debajo del umbral de los 5 segundos.
2. **Fase 2 – Ensemble Condicional Ligero (Smart Validation):** Aquí es donde el sistema empieza a «pensar» sobre su propia confianza. Se introduce el concepto de activación por umbrales:
   - Verificación por *Score*: El modelo «Juez» solo entra en juego si el *score* de similitud de **AlloyDB** es bajo (p. ej., < 0.75), optimizando el uso de la API de **Vertex AI**.
   - Clasificación de Riesgo: Se identifican consultas sobre políticas financieras o legales sensibles para forzar una doble verificación.

3. **Fase 3 – Ensemble Estratégico Completo (Gobernanza Total):** La fase final representa el estado del arte en sistemas RAG. Se integran capas de *Reranking* multi-modelo y se consolidan los validadores simbólicos que actúan como «guardias de seguridad» finales. El sistema es capaz de alternar dinámicamente entre RAG estándar y MAG (*Multi-Agent Generation*) para casos de uso de gran escala.

El Rol de *LangChain* como Catalizador

Es fundamental comprender que la adopción de *LangChain* no es un mero ejercicio de organización de código o «refactorización». Representa la creación de la base estructural necesaria para una arquitectura adaptable. Sin esta orquestación, cualquier intento de *ensemble* resultaría en un sistema rígido, costoso y difícil de mantener.

En el ecosistema de **Spin Compass**, el *ensemble* no se define por la redundancia permanente de modelos —lo cual sería financieramente insostenible— sino como un sistema de decisiones jerárquico. La inteligencia del sistema reside en saber cuándo ser simple y cuándo ser profundo.

Aplicación para SLACK

La arquitectura actual es la siguiente:

```
compass/
├─ agents/
│  ├─ chatbot_core.py
│  └─ __pycache__/
├─ api/
│  ├─ slack_bot_async.py
│  └─ __pycache__/
├─ config/
│  ├─ __init__.py
│  ├─ config.ini
│  ├─ settings.py
│  └─ __pycache__/
├─ database/
│  ├─ __init__.py
│  ├─ conversation_strategies.py
│  ├─ data_vault_manager.py
│  ├─ initial_load_compass.sql
│  ├─ recommended_indexes.sql
│  ├─ spin_compass_ddl.sql
│  ├─ spin_voyager_ddl.sql
│  └─ __pycache__/
├─ ingestion/
│  ├─ chunking/
│  ├─ drive/
│  └─ pipelines/
│     ├─ job_pdf.py
│     └─ pdf_processor.py
├─ prompts/
│  ├─ __init__.py
│  ├─ rag_prompt.py
│  └─ __pycache__/
├─ utils/
│  ├─ output_validator.py
│  ├─ pii_sanitizer.py
│  ├─ rag_models.py
│  ├─ rate_limiter.py
│  ├─ README.md
│  ├─ sensitive_detector.py
│  └─ __pycache__/
├─ .dockerignore
├─ .env
├─ app_async.py
├─ docker-compose.yml
├─ Dockerfile
├─ LOCAL_SETUP.md
├─ mna-proyecto-integrador-4440ad08...
└─ requirements.txt
```

Diagrama 1: Arquitectura de archivos del proyecto Spin Compass

Descripción de Componentes y Arquitectura

Entrada / API Principal (`app_async.py`) Punto de entrada de la aplicación que utiliza un enfoque asíncrono para gestionar las solicitudes de Slack, optimizando la concurrencia y los tiempos de respuesta del bot.

Interfaz de Mensajería (`api/slack_bot_async.py`) Actúa como el adaptador específico para la API de Slack. Gestiona la recepción de eventos, el parseo de payloads y el envío de respuestas de forma no bloqueante.

Procesamiento de Documentos (`ingestion/`)

- `pipelines/pdf_processor.py`: Orquesta la extracción de contenido desde archivos PDF.
- `chunking/`: Módulo encargado de la segmentación inteligente del texto para su posterior indexación en el motor de búsqueda vectorial.

Núcleo RAG (`agents/chatbot_core.py`)

- Orquesta el flujo completo de recuperación y generación (RAG).
- Coordina la interacción con `Vertex AI` para la generación de respuestas y la lógica de búsqueda semántica.

Capa de Persistencia y SQL (`database/`)

- `data_vault_manager.py`: Implementa el patrón **Data Vault 2.0** para el almacenamiento robusto de interacciones.
- **Scripts DDL** (`.sql`): Define los esquemas para Spin Compass y Spin Voyager, asegurando la consistencia de datos e índices.

Seguridad y Validación (`utils/`)

- `pii_sanitizer.py` y `sensitive_detector.py`: Componentes críticos de cumplimiento que filtran información sensible o personal (PII).
- `output_validator.py`: Valida que las respuestas del modelo cumplan con los criterios de calidad y formato.

- `rate_limiter.py`: Controla el flujo de peticiones para evitar saturación de las APIs externas.

Configuración y Prompts

- `config/settings.py`: Carga centralizada de variables de entorno (`.env`) y parámetros de sistema (`config.ini`).
- `prompts/rag_prompt.py`: Repositorio de plantillas de sistema para asegurar la consistencia en las respuestas del agente.

Flujo de Datos (Alto Nivel)

1. **Evento Slack** → `api/slack_bot_async.py` (vía `app_async.py`)
2. **Sanitización PII** → `utils/pii_sanitizer.py` (Limpieza de entrada)
3. **Consulta Core** → `agents/chatbot_core.py`
4. **Historial y Contexto** → Recuperación desde `DataVaultManager` y `AlloyDB`
5. **Generación** → `ChatVertexAI` (Gemini 2.0 Flash-Lite) con `rag_prompt.py`
6. **Validación de Salida** → `utils/output_validator.py` y `sensitive_detector.py`
7. **Persistencia** → `database/data_vault_manager.py` (Esquema Data Vault 2.0)
8. **Respuesta** → Envío asíncrono a Slack y control por `rate_limiter.py`

Estatus Actual

`chatbot_core.py` (Core RAG) Implementa la lógica de recuperación y generación apoyada en el historial conversacional. Utiliza orquestación avanzada para `Vertex AI` y gestiona la búsqueda semántica en `AlloyDB`. Incluye el manejo de sesiones y persistencia, optimizado para el flujo de trabajo del proyecto Spin Compass.

`data_vault_manager.py` (Persistencia) Adaptador integral para un esquema **Data Vault 2.0** sobre `AlloyDB`. Gestiona de forma granular usuarios, conversaciones y feedback, permitiendo estrategias de sesión parametrizables. Es un componente robusto que

encapsula la lógica de negocio de la base de datos mediante los scripts DDL incluidos en el módulo.

Integración con Slack (Asíncrona)

- `app_async.py`: Orquesta la comunicación mediante el modo asíncrono de `Slack Bolt`.
- `api/slack_bot_async.py`: Actúa como el adaptador de eventos moderno para la interfaz de usuario en Slack, mejorando la escalabilidad.

Configuración y Utilidades

- `settings.py`: Centraliza la configuración mediante una `dataclass` que valida variables de entorno y archivos `.ini`.
- `utils/`: El proyecto ha madurado incluyendo validadores de salida (`output_validator.py`) y herramientas de limpieza de datos personales (`pii_sanitizer.py`).

Infraestructura y Despliegue Soporte nativo para contenedores (`Dockerfile`) y orquestación con `docker-compose.yml`. Preparado para despliegue en `Cloud Run` o `Kubernetes`, aprovechando los **healthchecks** y la configuración de entorno definida.

Pruebas de Funcionalidad de Compass

Evaluación del Conocimiento del Agente

Para iniciar las pruebas funcionales del bot, se definió como primer objetivo evaluar su capacidad de conocimiento y alcance operativo. En este caso, se buscó identificar qué tipo de servicios y apoyo puede proporcionar al empleado.

Se realizó la pregunta:

«Hola Compass, ¿qué puedes hacer?»

Compass respondió con una descripción clara de su rol, indicando que es un asistente financiero de Spin y que puede apoyar respondiendo preguntas relacionadas con políticas corporativas.

Adicionalmente, proporcionó un listado de los documentos a los que tiene acceso y sobre los cuales puede realizar consultas.

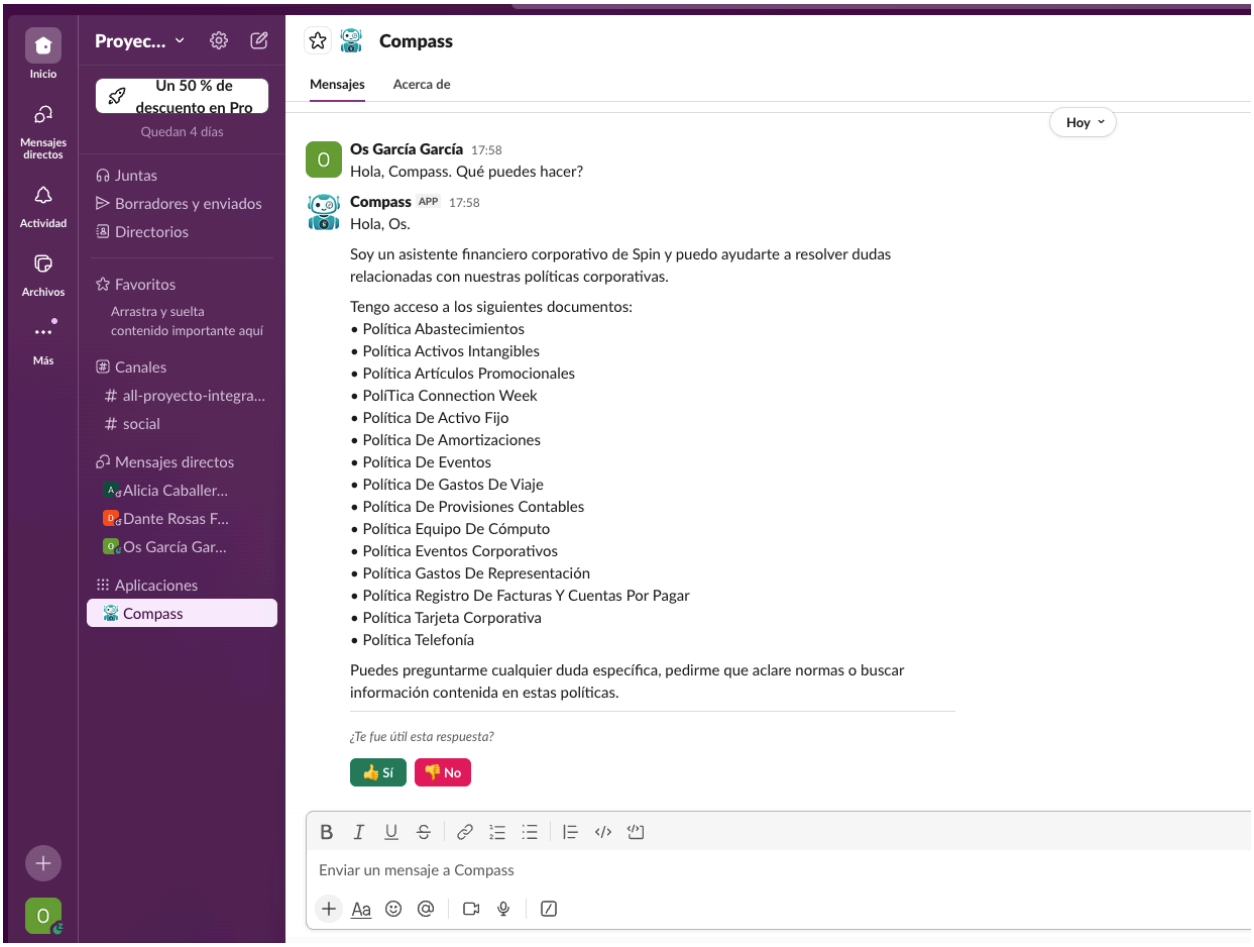


Figura 1: Lista informativa de Compass

Consulta sobre Políticas de Viáticos

Consulta Inicial

Se realizó una consulta relacionada con los montos establecidos para alimentos durante viajes corporativos.

Compass responde utilizando el nombre registrado en Slack y solicita información adicional para brindar una respuesta más precisa, ya que los montos varían según:

- Tipo de viaje (nacional o internacional)
- Tipo de puesto o grado

Posteriormente menciona el documento y sección consultada, así como una explicación general sobre la acumulación de gastos, requisitos de reembolso y conceptos no reembolsables.

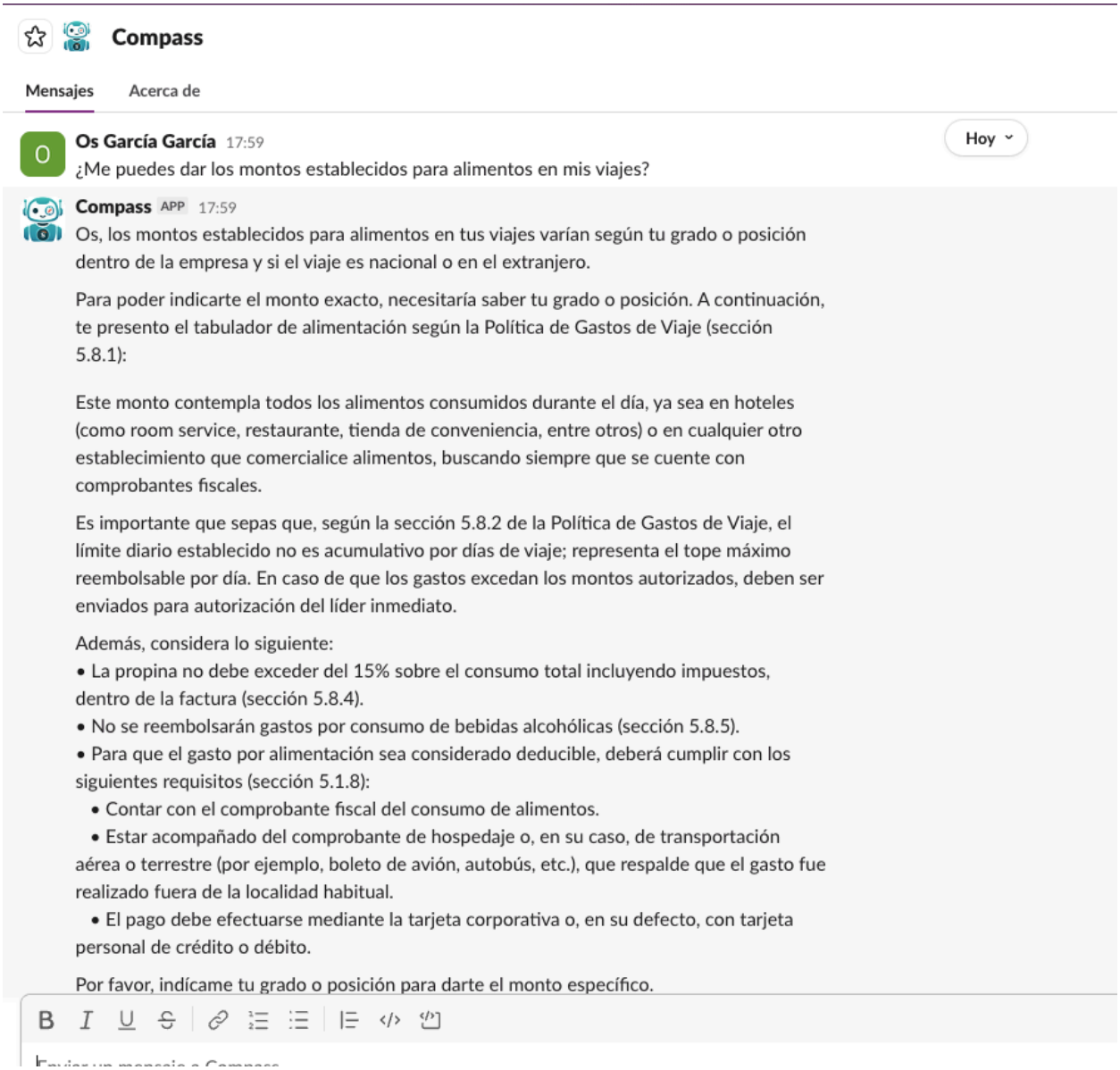


Figura 2: Consulta sobre montos de alimentos en viajes

Aclaración del Puesto

Una vez indicado que el puesto corresponde a colaborador, Compass proporciona los montos máximos para viajes nacionales e internacionales, incluyendo una tabla con el desglose por tipo de puesto.

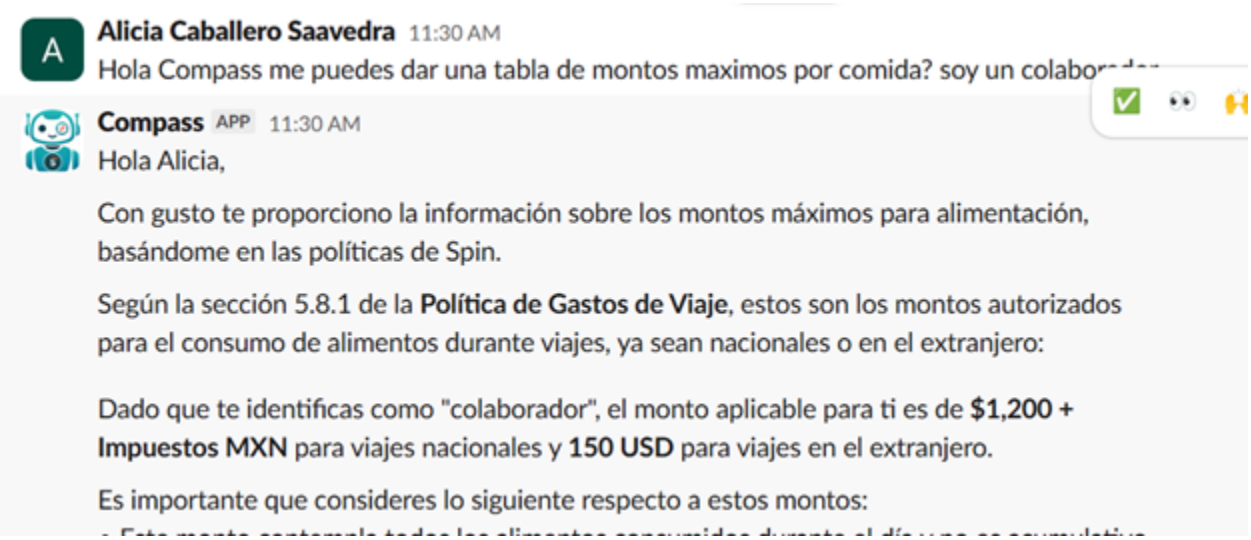
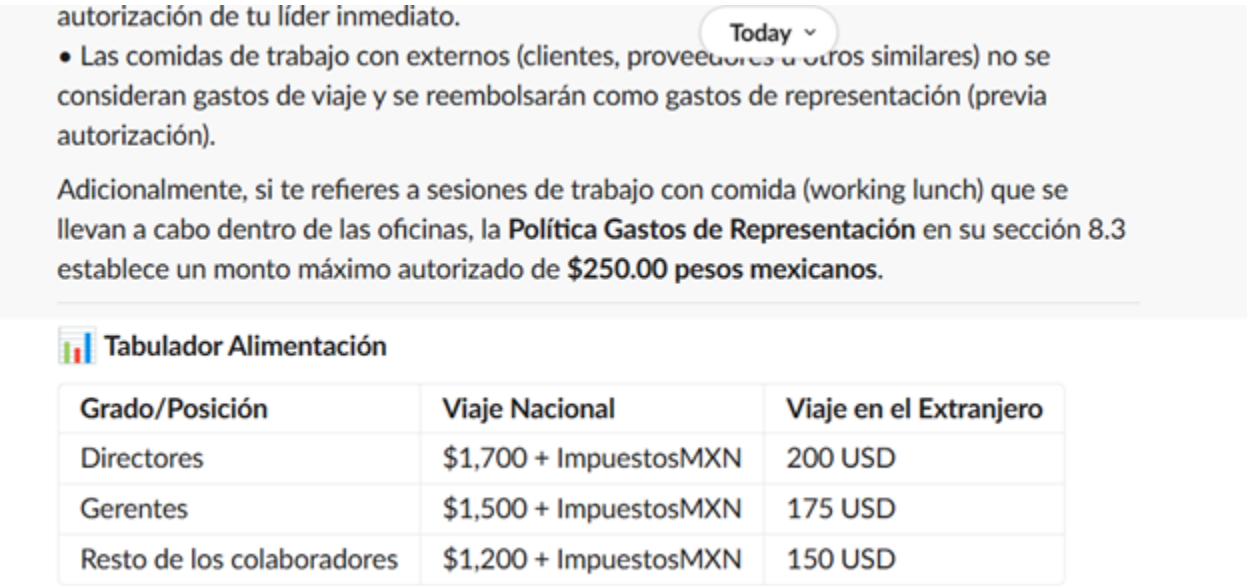


Figura 3: Hilo de conversación 1



Grado/Posición	Viaje Nacional	Viaje en el Extranjero
Directores	$1,700 + ImpuestosMXN	200 USD
Gerentes	$1,500 + ImpuestosMXN	175 USD
Resto de los colaboradores	$1,200 + ImpuestosMXN	150 USD

Figura 4: Hilo de conversación 1 - continuación

Consulta sobre Monto Máximo de Propina

Se realizó la pregunta sobre el porcentaje máximo permitido para propina. Compass respondió correctamente que el monto máximo permitido es del 15%, mencionando el documento y sección correspondiente.

Figura 5: Consulta sobre monto máximo de propina

Validación de Enlaces y Anexos

Se realizó la pregunta:

«¿Cuáles son los hoteles en convenio para una Connection Week?»

Compass indicó la política y anexo correspondiente y proporcionó el enlace al documento alojado en Drive, el cual fue validado y se encuentra funcionando correctamente.

Figura 6: Validación de enlaces y anexos

Prueba de Privacidad y Acceso a Información Personal

Se solicitó información del organigrama con el objetivo de validar restricciones de acceso a datos personales.

Compass aclaró que no tiene acceso a información personal y orientó al usuario hacia el departamento correspondiente.

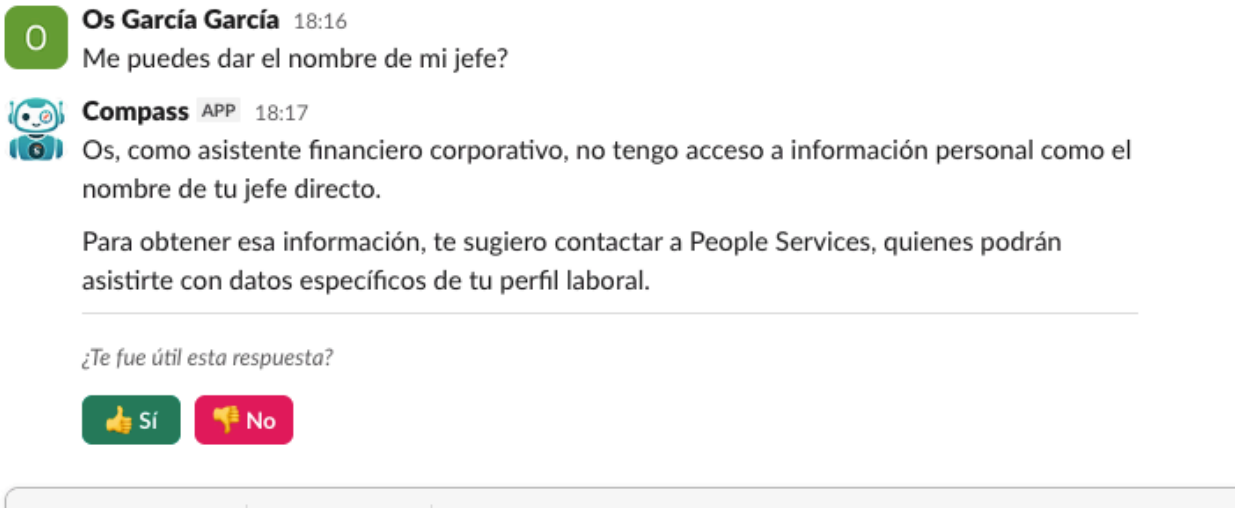


Figura 7: Privacidad de Datos

Sistema de Feedback

Feedback Positivo

Cuando el usuario selecciona que la respuesta fue útil, el sistema agradece la retroalimentación y no realiza acciones adicionales.

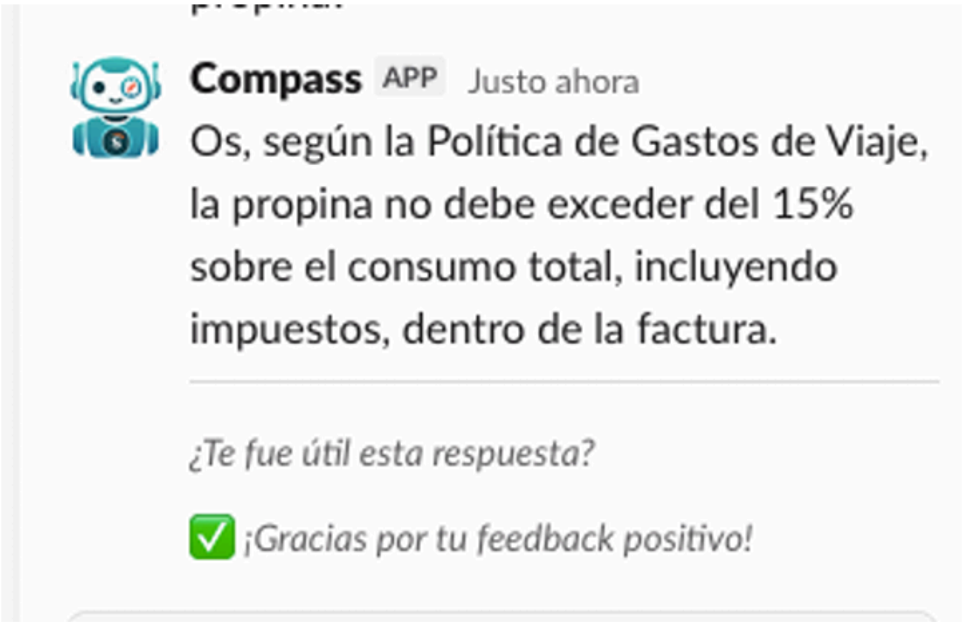


Figura 8: Feedback Positivo

Feedback Negativo

Si el usuario selecciona que la respuesta no fue útil, se despliega una ventana emergente que permite registrar comentarios adicionales.

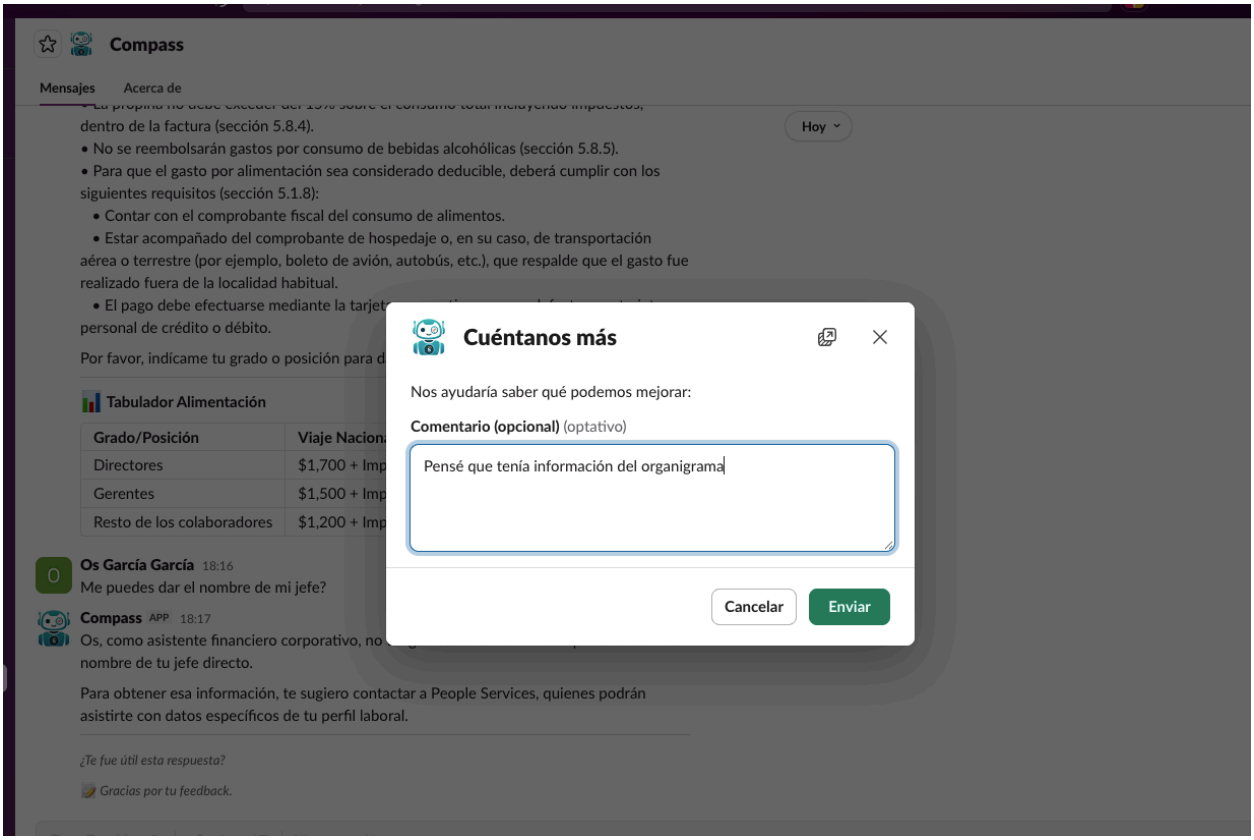


Figura 9: Ventana emergente de comentario negativo

Posteriormente, Compass agradece el comentario y este queda almacenado para revisión administrativa.

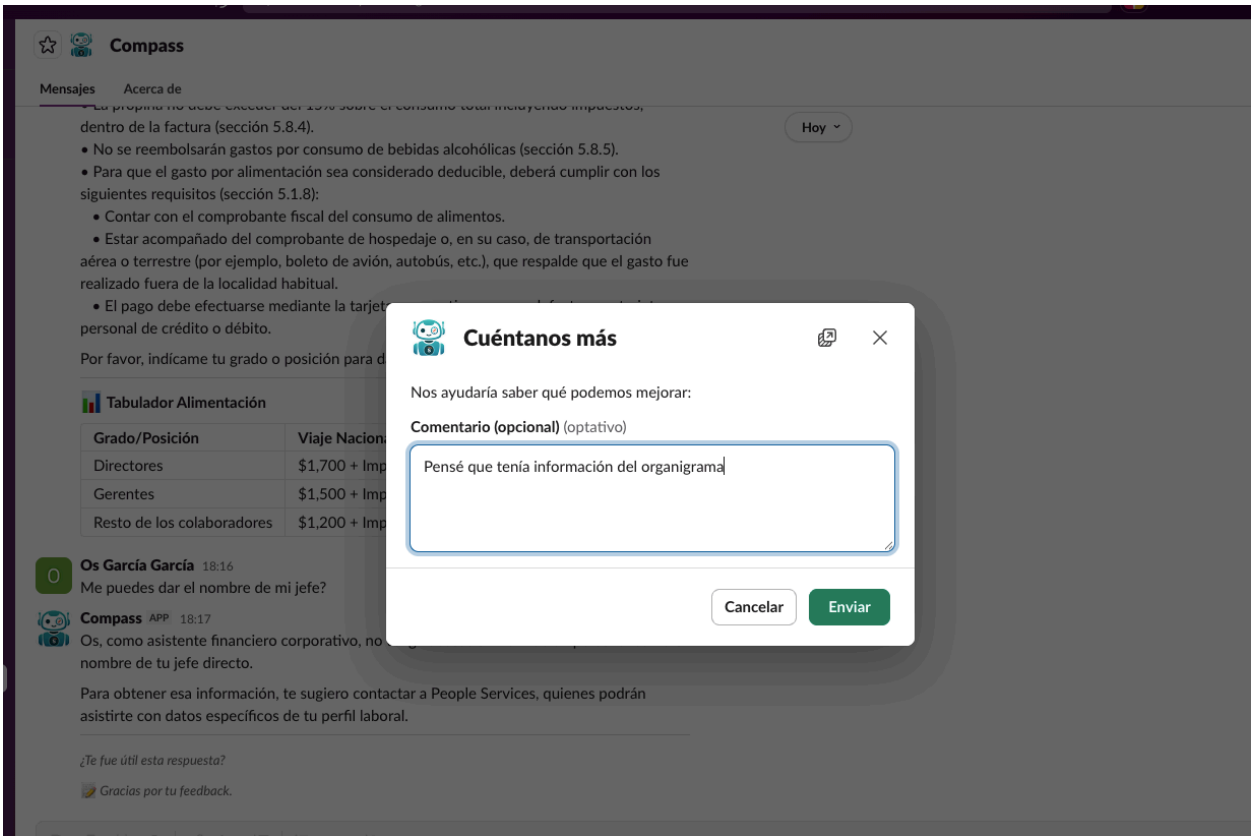


Figura 10: Feedback Negativo

Funcionalidades del Administrador

Configuración del Agente en Slack

En la sección de configuración básica se pueden modificar:

- Nombre
- Descripción
- Colores
- Logo
- Alcances (Scopes)
- Secrets

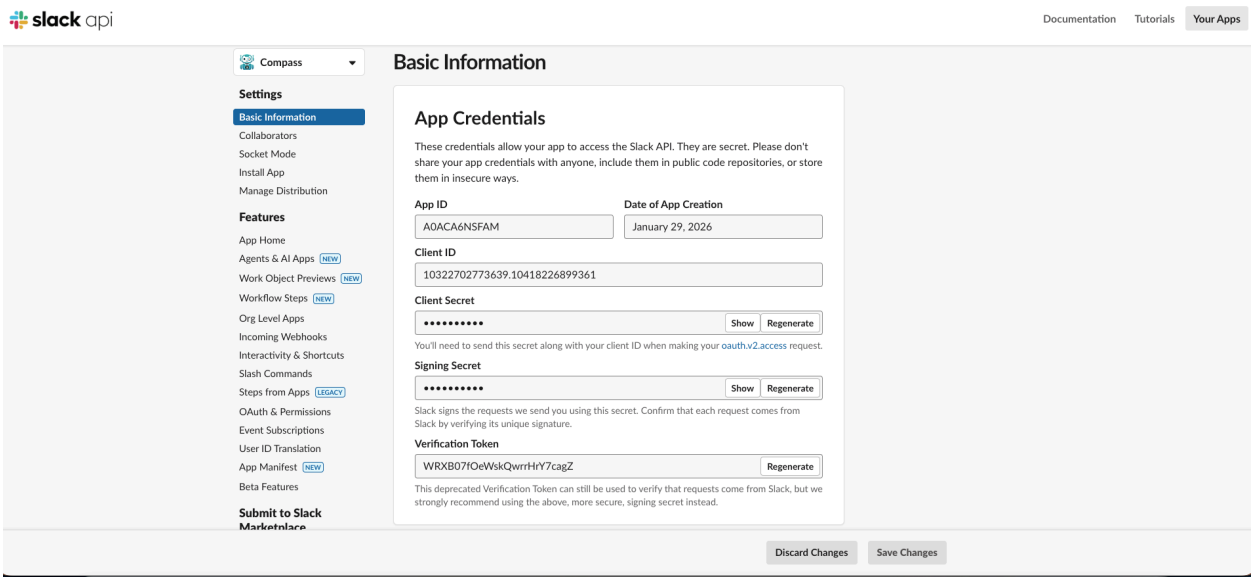


Figura 11: Configuración del Agente

Configuración de Ámbitos (Scopes) de la API

Los permisos configurados incluyen:

- chat:write
- chat:write.public
- reactions:read
- users:profile:read
- users:read
- users:read.email

Ámbitos

Las capacidades y los permisos de una aplicación de Slack se rigen por los ámbitos que solicita.

Ámbitos de los tokens de usuarios bot

Ámbitos que rigen a qué puede acceder tu aplicación.

Ámbito de OAuth	Descripción
app_mentions:read	Ver mensajes que mencionan directamente @Compass en conversaciones en las que se encuentra la aplicación
chat:write	Enviar mensajes como @Compass
chat:write.public	Enviar mensajes a canales de los que @Compass no es miembro
im:history	Ver mensajes y otro contenido de los mensajes directos a los que se haya añadido "Compass"
reactions:read	Ver las reacciones emoji y su contenido asociado de los canales y conversaciones a los que se haya añadido "Compass"
reactions:write	Añadir y editar reacciones emoji
users.profile:read	Ver los datos de perfil de los miembros de un espacio de trabajo
users:read	Ver miembros de un espacio de trabajo
users:read.email	Ver las direcciones de correo electrónico de los miembros de un espacio de trabajo

Figura 12: API – Alcances del bot

Spin Bridge – Dashboard Ejecutivo

Spin Bridge funciona como interfaz entre Compass y el administrador, permitiendo la visualización de métricas y desempeño del agente.

Dashboard Principal

Permite visualizar métricas en periodos de 7 o 30 días:

- Número de conversaciones
- Porcentaje de feedback
- Número de usuarios activos
- Tasa de fallback
- Tendencia semanal de mensajes

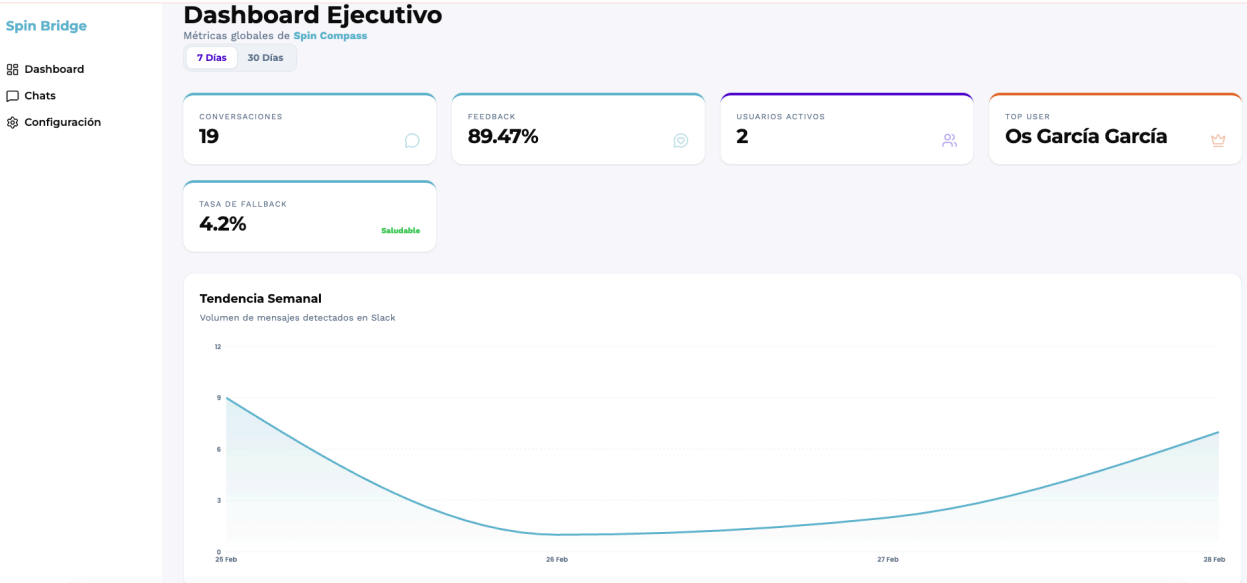


Figura 13: Dashboard Spin Bridge

Análisis de Conversaciones con Feedback Negativo

En la sección de feedback se muestran las respuestas que generaron retroalimentación negativa y el contexto completo de la conversación.

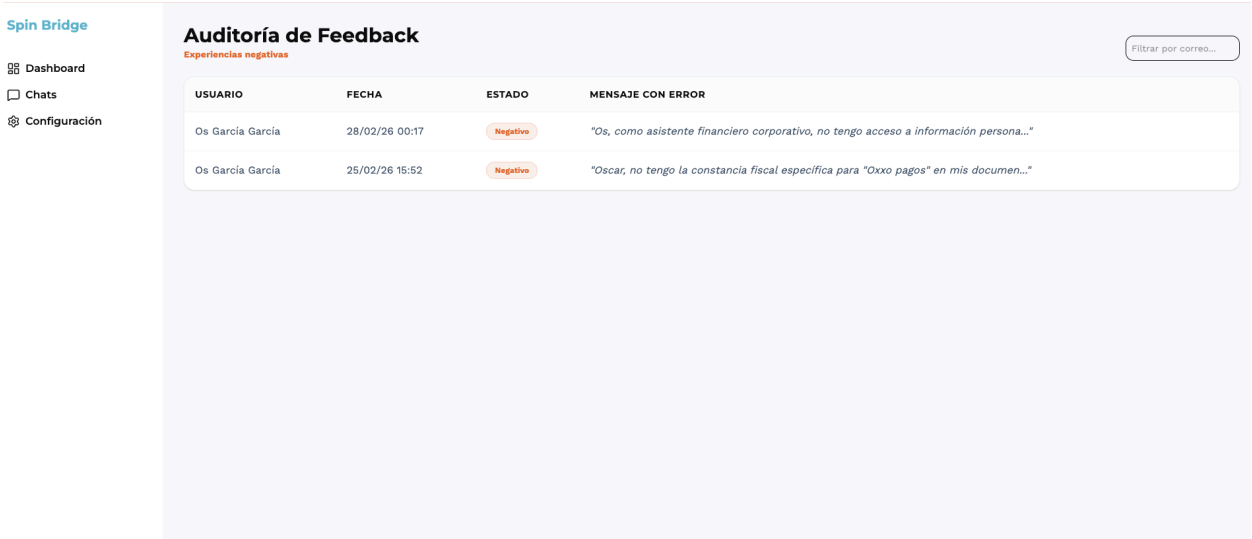


Figura 14: Página de análisis de conversaciones

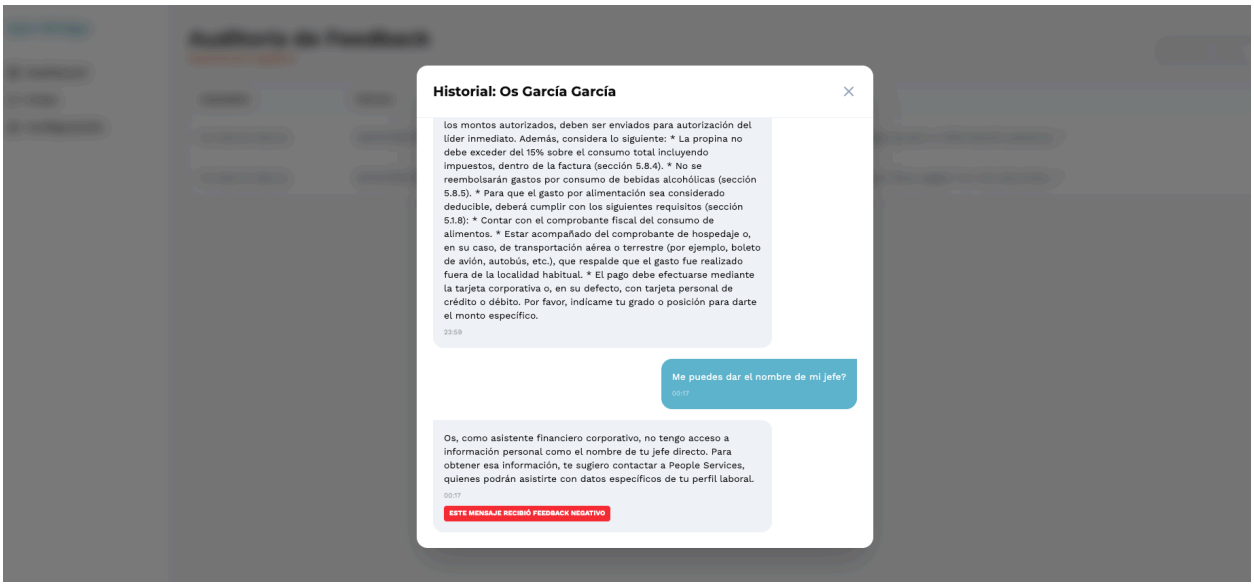


Figura 15: Detalle de conversación con feedback negativo

Configuración Avanzada del Agente (En Desarrollo)

Se encuentra en desarrollo una ventana que permitirá editar el prompt del agente y aplicar cambios dinámicos.

Estado del Proyecto y conclusiones

El dashboard ejecutivo se encuentra actualmente en construcción. En futuras entregas se incorporarán mejoras visuales, métricas avanzadas y nuevas funcionalidades administrativas.

El equipo coincide en que existen múltiples configuraciones aplicables para el modelo LLM con RAG. Como se especificó en el apartado “Posibles configuraciones de Ensembles aplicables”, existen arquitecturas que optimizan significativamente los resultados; no obstante, debido a las restricciones presupuestarias del patrocinador y a los tiempos de entrega definidos, estas no pudieron incluirse en el alcance (scope) inicial para su correcto desarrollo, prueba y ejecución.

Bibliografía

[1] L. Visengeriyeva, A. Kammer, I. Bär, A. Kniesz, y M. Plöd, «CRISP-ML(Q): The ML lifecycle process». Accedido: 22 de enero de 2026. [En línea]. Disponible en: https://ml-ops.org/content/crisp-ml

[2] R. Hernández-Sampieri y C. Mendoza, *Metodología de la investigación: Las rutas de la investigación cuantitativa, cualitativa y mixta*, 3.ª ed. McGraw-Hill, 2023.

[3] A. Moreno-Cediel, E. Garcia-Lopez, D. De-Fitero-Dominguez, y A. Garcia-Cabot, «Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries». Alcalá de Henares, Madrid, Spain, 2025.

[4] C. Kiss, M. Nagy, y P. Szilagyi, «Max–Min semantic chunking of documents for RAG application». Budapest, Hungary, 2025.

[5] A. Singh, «Comprehensive Guide to Ensemble Learning (with Python codes)». [En línea]. Disponible en: https://www.analyticsvidhya.com/blog/2018/06/comprehensive-guide-for-ensemble-models/#4

[6] J. VanderPlas, «Thinking about model validation», en *Python Data Science Handbook*, 2nd ed., O'Reilly Media, 2022, chap. 39. [En línea]. Disponible en: https://learning.oreilly.com/library/view/python-data-science/9781098121211/

[7] L. Breiman, «Bagging predictors», *Machine Learning*, vol. 24, n.º 2, pp. 123-140, 1996.

[8] T. G. Dietterich, «Ensemble methods in machine learning», *Multiple Classifier Systems*, pp. 1-15, 2000.

[9] P. Lewis *et al.*, «Retrieval-augmented generation for knowledge-intensive NLP tasks», *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020.

[10] Y. Gao, W. Xiong, X. Li, Z. Liu, T. Liu, y E. Xing, «Retrieval-augmented generation for large language models: A survey», *arXiv preprint arXiv:2312.10997*, 2023.

[11] K. Shuster, J. Xu, D. J. Smith, y J. Weston, «Retrieval augmentation reduces hallucination in conversation», *Findings of EMNLP*, 2021.

[12] LangChain, «LangChain Documentation». 2023.

[13] Google Cloud, «Vertex AI documentation». 2024.

[14] X. Wang y others, «Self-refine: Iterative refinement with self-feedback», *arXiv preprint arXiv:2303.17651*, 2023.